7 - Resolução de Problemas e Algoritmos

No Capítulo 6, examinamos o código de máquina, que usa números binários para representar operações, e a linguagem assembly, que usa mnemônicos para representar operações. As linguagens assembly são um passo na direção certa, mas o programador ainda precisa pensar em termos de instruções de máquina individuais. Também introduzimos o pseudocódigo, uma linguagem artificial projetada para expressar algoritmos. Começamos este capítulo com uma breve discussão sobre resolução de problemas em geral.

A ciência da computação é, às vezes, definida como "o estudo de algoritmos e sua implementação eficiente em um computador". O foco deste capítulo são os algoritmos: seu papel na resolução de problemas, estratégias para desenvolvê-los, técnicas para segui-los e testá-los. Escolhemos algoritmos clássicos de busca e ordenação como contexto para a discussão sobre algoritmos.

Como os algoritmos operam com dados, examinamos maneiras de estruturá-los para que possam ser processados com mais eficiência.

OBJETIVOS

Após estudar este capítulo, você deverá ser capaz de:

- descrever o processo de resolução de problemas computacionais e relacioná-lo à lista "Como Resolver" de Polya.
- distinguir entre um tipo simples e um tipo composto.
- descrever três mecanismos compostos de estruturação de dados.
- reconhecer um problema recursivo e escrever um algoritmo recursivo para resolvê-lo.
- distinguir entre um array não ordenado e um array ordenado.
- distinguir entre uma ordenação por seleção e uma ordenação por inserção.
- descrever o algoritmo Quicksort.
- aplicar manualmente a ordenação por seleção, a ordenação por bolha, a ordenação por inserção e o Quicksort a um array de itens.
- aplicar o algoritmo de busca binária.
- demonstrar sua compreensão dos algoritmos deste capítulo simulando-os manualmente com uma sequência de itens.

7.1 Como Resolver Problemas

Em 1945, George Polya escreveu um pequeno livro intitulado "Como Resolver: um novo aspecto do método matemático".[1] Embora este livro tenha sido escrito há mais de 60 anos, quando os computadores ainda eram experimentais, ele continua sendo a descrição clássica do processo de resolução de problemas. O processo está resumido na FIGURA 7.1 (tradução nossa).

COMO RESOLVER	
Primeiro	**ENTENDENDO O PROBLEMA**
Você precisa entender o problema.	Qual é o desconhecido (ou a incógnita)? Quais são os dados? Qual é a condição?
	É possível satisfazer a condição? A condição é suficiente para determinar a incógnita? Ou é insuficiente? Ou redundante? Ou contraditória?
	Desenhe uma figura. Introduza a notação adequada.
	Separe as várias partes da condição. Você consegue anotá-las?
Segundo	**ELABORANDO UM PLANO**
Encontre a conexão entre os dados e o desconhecido. Você pode ser obrigado a considerar problemas auxiliares se uma conexão imediata não puder ser encontrada. Você deve obter eventualmente um plano de solução.	Você já viu isso antes? Ou já viu o mesmo problema de uma forma ligeiramente diferente?
	Você conhece algum problema relacionado? Você conhece algum teorema que poderia ser útil? Observe a incógnita! E tente pensar em um problema familiar que tenha a mesma incógnita ou uma incógnita semelhante.
	Aqui está um problema relacionado ao seu e já visto antes. Você poderia usá-lo? Você poderia usar o resultado? Você poderia usar o método 1? Você deveria introduzir algum elemento auxiliar para tornar seu uso possível? Você poderia reformular o problema? Poderia reformulá-lo de forma diferente? Voltar para as definições.
	Se você não conseguir resolver o problema proposto, tente resolver primeiro algum problema relacionado. Você consegue imaginar um problema relacionado mais acessível? Um problema mais geral? Um problema mais específico? Um problema análogo? Você conseguiria resolver uma parte do problema? Manter apenas uma parte da condição, eliminar a outra parte; até

	que ponto a incógnita é então determinada, como ela pode variar? Você conseguiria derivar algo útil dos dados? Você conseguiria pensar em outros dados apropriados para determinar a incógnita? Você conseguiria alterar a incógnita ou os dados, ou ambos, se necessário, para que a nova incógnita e os novos dados fiquem mais próximos um do outro? Você utilizou todos os dados? Você utilizou a conclusão completa? Você levou em consideração todas as noções essenciais envolvidas no problema?
Terceiro	**EXECUÇÃO DO PLANO**
Execute seu plano.	Executando seu plano de solução, verifique cada etapa. Você consegue ver claramente que a etapa está correta? Você consegue provar que está correta?
Quarto	**REVISÃO**
Examine a solução obtida.	Você consegue verificar o resultado? Você consegue verificar o argumento? Você consegue derivar o resultado de forma diferente? Você consegue vê-lo de relance? Você consegue usar o resultado, ou o método, para algum outro problema?

FIGURA 7.1 Lista de "How to Solve It" (ou Como Resolver) de Polya

Reproduzido de POLYA, G.; How to Solve It. © 1945 Princeton University Press, 1973, versão renovada do PUP. Reproduzido com permissão da Princeton University Press.

O que tornou o livro de Polya um clássico é que sua lista de "Como Resolver" é bastante geral. Embora tenha sido escrita no contexto da resolução de problemas matemáticos, podemos substituir as palavras "desconhecido" ou "incógnita" por "problema", "dados" por "informação" e "teorema" por "solução", e a lista se torna aplicável a todos os tipos de problemas. Claro, é o segundo passo – encontrar a conexão entre a informação e a solução – que está no cerne da resolução de problemas. Vejamos algumas estratégias sugeridas pela lista de Polya.

Faça Perguntas

Se você recebe um problema ou tarefa verbalmente, normalmente faz perguntas até que o que deve fazer fique claro. Você pergunta **quando**, **por que** e **onde** até que sua tarefa esteja completamente especificada. Se suas instruções forem escritas, você pode colocar pontos de interrogação na margem; sublinhar uma palavra, um grupo de palavras ou uma frase; ou de alguma outra forma indicar as partes da tarefa que não estão claras. Talvez suas perguntas possam ser respondidas em um parágrafo posterior, ou você pode ter que discuti-las com a pessoa que lhe deu a tarefa. Se a tarefa for definida por você, esse tipo de questionamento pode não ser verbal, mas sim ocorrer no nível subconsciente.

Algumas perguntas típicas que você deve fazer são as seguintes: *O que eu sei sobre o problema? Como é a solução? Que tipos de casos especiais existem? Como reconhecerei que encontrei a solução?*

Procure por Coisas Familiares

Você nunca deve reinventar a roda. Se houver uma solução, use-a. Se você já resolveu o mesmo problema ou um problema semelhante antes, apenas repita a solução bem-sucedida. Normalmente, não pensamos conscientemente: "Já vi isso antes e sei o que fazer" – simplesmente fazemos. Os humanos são bons em reconhecer situações semelhantes. Não precisamos aprender a ir à loja para comprar leite, depois a comprar ovos e depois a comprar doces. Sabemos que ir à loja é sempre a mesma coisa e apenas o que compramos é diferente.

Reconhecer situações familiares é particularmente útil em computação. Na computação, você vê certos problemas repetidamente, sob diferentes formas. Um bom programador vê uma tarefa, ou talvez parte de uma tarefa (uma subtarefa), que já foi resolvida antes e insere a solução. Por exemplo, encontrar as temperaturas máximas e mínimas diárias em uma lista de temperaturas é exatamente o mesmo problema que encontrar as notas mais altas e mais baixas em uma lista de notas de teste. Você precisa dos maiores e menores valores em um conjunto de números.

Dividir e Conquistar

Constantemente, dividimos um grande problema em unidades menores que possamos resolver. A tarefa de limpar a casa ou o apartamento, por exemplo, pode parecer esmagadora. Em contraste, as tarefas individuais de limpar a sala de estar, a sala de jantar, a cozinha, os quartos e o banheiro parecem mais administráveis. Este princípio é especialmente relevante para computação: dividimos um grande problema

em partes menores, e dividimos cada problema menor em outros problemas menores ainda até que possamos resolver cada subproblema individualmente.

Essa abordagem aplica o conceito de abstração que discutimos no Capítulo 1 – limpar a casa é um problema amplo e abstrato, composto por subtarefas definidas pela limpeza dos cômodos individuais. Limpar um cômodo também pode ser considerado uma abstração dos detalhes de arrumar a cômoda, arrumar a cama, aspirar o chão e assim por diante. As tarefas são divididas em subtarefas, que podem ser subdivididas em subsubtarefas e assim por diante. A abordagem de dividir para conquistar pode ser aplicada repetidamente até que cada subtarefa seja gerenciável, solucionável facilmente.

Aplicamos duas dessas estratégias no último capítulo, quando fizemos perguntas e adiamos detalhes ao projetar o algoritmo para ler dois números e exibi-los em ordem.

Algoritmos

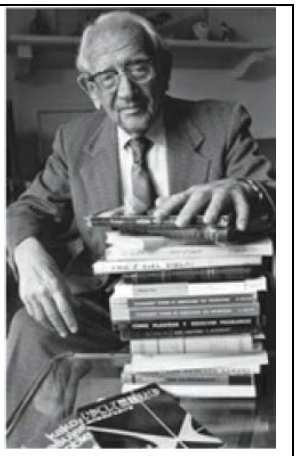
A última frase da segunda etapa da lista de Polya diz que você deve, eventualmente, obter um plano da solução. Em computação, esse plano é chamado de algoritmo. Usamos o termo muitas vezes; aqui o definimos em termos de computação. Formalmente, um algoritmo é um conjunto de instruções para resolver um problema ou subproblema em um período de tempo finito, usando uma quantidade finita de dados. Implícito nessa definição está o entendimento de que as instruções são inequívocas.

*Algoritmo: Instruções inequívocas para resolver um problema ou subproblema bem definido em um período de tempo finito, usando uma quantidade finita de dados.

Em computação, devemos explicitar certas condições que estão implícitas em soluções humanas. Por exemplo, na vida cotidiana, não consideraríamos uma solução que levasse uma eternidade como uma solução considerável. Também rejeitaríamos uma solução que exija o processamento de mais informações do que somos capazes de processar. Essas restrições devem ser explícitas em uma solução computacional, portanto, a definição de um algoritmo as inclui.

O terceiro passo na lista de Polya é executar o plano – ou seja, testar a solução para ver se ela resolve o problema. O quarto passo é examinar a solução para aplicabilidade futura.

George Polya



© AP Photos

George Polya nasceu em Budapeste em 13 de dezembro de 1887. Embora tenha se tornado um matemático mundialmente famoso, não demonstrou interesse precoce por matemática. A falta de interesse de Polya pode ser explicada por sua lembrança de três professores de matemática do ensino médio: "dois eram desprezíveis e um era bom".

Em 1905, Polya ingressou na Universidade de Budapeste, onde estudou direito por insistência de sua mãe. Depois de um semestre muito chato, decidiu estudar línguas e literatura. Obteve um certificado de professor de latim e húngaro — e nunca o utilizou. Interessou-se por filosofia e fez cursos de matemática e física como parte de seus estudos de filosofia. Decidiu-se pela matemática, comentando: "Sou bom demais para a filosofia e não o suficiente para a física. A matemática está no meio termo". Obteve seu doutorado em matemática em 1912, o que lançou sua carreira.

Polya pesquisou e lecionou na Universidade de Göttingen, na Universidade de Paris e na Federação Suíça de Tecnologia em Zurique. Enquanto estava em Zurique, ele interagiu com John von Neumann, sobre quem disse: "Johnny era o único aluno de quem eu tinha medo. Se, durante uma palestra, eu apresentasse um problema não resolvido, era provável que ele viesse até mim assim que a palestra terminasse, com a solução completa em alguns rabiscos em um pedaço de papel."

Como muitos europeus da época, Polya mudou-se para os Estados Unidos em 1940 devido à situação política na Alemanha. Após lecionar na Universidade Brown por dois anos, mudou-se para Palo Alto, Califórnia, para lecionar na Universidade Stanford, onde permaneceu pelo resto de sua carreira.

As pesquisas e publicações de Polya abrangeram diversas áreas da matemática, incluindo teoria dos números, combinatória, astronomia, probabilidade, funções integrais e problemas de valor de contorno para equações diferenciais parciais. O Prêmio George Polya é concedido em sua homenagem por aplicações notáveis da teoria combinatória.

No entanto, apesar de todas as contribuições de George Polya à matemática, é sua contribuição para o ensino de matemática da qual ele mais se orgulha e pela qual será mais lembrado. Seu livro "How to Solve It", publicado em 1945, vendeu mais de 1 milhão de cópias e foi traduzido para 17 idiomas. Neste livro, Polya descreve uma estratégia de resolução de problemas projetada para problemas matemáticos. No entanto, a generalidade da estratégia a torna aplicável a todas as resoluções de problemas. A estratégia de Polya é a base da estratégia de resolução de problemas computacionais descrita neste texto. Mathematics and Plausible Reasoning (ou Matemática e Raciocínio Plausível), publicado em 1954, foi outro livro dedicado à educação matemática. Polya não apenas escreveu sobre educação matemática, mas também se interessou ativamente pelo ensino de matemática. Ele era um visitante regular das escolas da região da Baía de São Francisco e visitava a maioria das faculdades nos estados do oeste. O Centro de Matemática da Universidade de Idaho leva seu nome.

Em 7 de setembro de 1985, George Polya faleceu em Palo Alto, aos 97 anos.

Processo de Resolução de Problemas Computacionais

Para agora, o processo de resolução de problemas computacionais inclui, basicamente, quatro fases: a fase de **análise e especificação**, a fase de **desenvolvimento do algoritmo**, a fase de **implementação do código** e a fase de **manutenção**. Veja a FIGURA 7.2. A saída da primeira fase é uma declaração de problema claramente escrita. A saída da fase de desenvolvimento do algoritmo é um plano para uma solução geral para o problema especificado na primeira fase. A saída da terceira fase é um programa de computador funcional que implementa o algoritmo – ou seja, uma solução específica para o problema. Não há saída da quarta fase, a menos que erros sejam detectados ou alterações precisem ser feitas. Nesse caso, esses erros ou alterações são enviados de volta para a primeira, segunda ou terceira fase, conforme o caso.

Fase de Análise e Especificação	
Análise	Entenda (definir) o problema.
Especificação	Especifique o problema que o programa resolverá.
Fase de Desenvolvimento do Algoritmo	
Desenvolver algoritmo	Desenvolva uma sequencia lógica de passos a ser usada para resolver o problema.
Testar algoritmo	Siga as etapas descritas para verificar se a solução realmente resolve o problema.
Fase de Implementação	
Codificar	Traduza o algoritmo para uma linguagem de programação.
Testar	Faça com que o computador siga as instruções. Verifique os resultados e faça correções até que as respostas estejam corretas.
Fase de Manutenção	
Uso	Use o programa.
Manutenção	Modifique o programa para atender às mudanças nos requisitos ou para corrigir eventuais erros.

FIGURA 7.2 O processo de resolução de problemas em computador.

A FIGURA 7.3 (ao lado) mostra como essas fases interagem. As linhas pretas mostram o fluxo geral através da fase. As linhas vermelhas representam caminhos que podem ser seguidos para retornar a uma fase anterior caso ocorra um problema. Por exemplo, durante o processo de produção de um algoritmo, pode ser encontrado um erro ou contradição na especificação, caso em que a análise e a especificação devem ser corrigidas. Da mesma forma, um erro no programa pode indicar que um algoritmo precisa ser corrigido.

Todas as fases de Polya estão incluídas neste esboço de como resolvemos problemas usando o computador. O primeiro passo é sempre entender o problema. Você não pode escrever uma solução computacional para um problema que você não entende. O próximo passo é desenvolver um plano – um algoritmo – para a solução e expressá-lo em pseudocódigo. É nesta fase que nos concentramos neste capítulo.

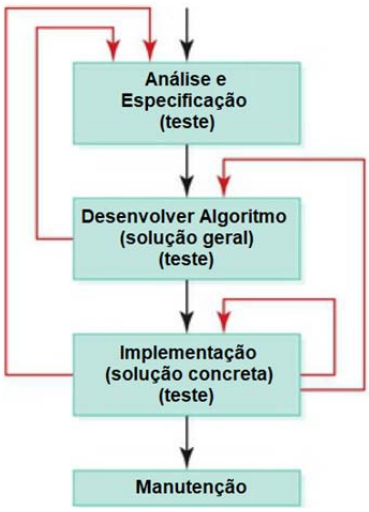


FIGURA 7.3 As interações entre as quatro fases de resolução de problemas.

O próximo passo é implementar o plano de forma que o computador possa executá-lo e testar os resultados. Na lista de Polya, o humano executa o plano e avalia os resultados. Em uma solução computacional, um programa é escrito expressando o plano em uma linguagem que o computador pode executar. Mas é o humano que pega os resultados do programa de computador e os verifica para confirmar se estão corretos. A fase de manutenção mapeia para o último estágio de Polya, onde os resultados são examinados e talvez modificados.

Neste capítulo, encerramos o processo no nível do pseudocódigo. Este texto é neutro em termos de linguagem; ou seja, não abordamos uma linguagem de alto nível em detalhes. No entanto, alguns de vocês podem estar aprendendo uma paralelamente a este texto. De qualquer forma, lembre-se de que um algoritmo deve ser escrito antes que qualquer codificação em uma linguagem de programação possa ser feita.

Resumo da Metodologia

A metodologia para projetar algoritmos pode ser dividida em quatro etapas principais:

1. Analise o problema

Entenda o problema! Liste as informações com as quais você precisa trabalhar. Essas informações provavelmente consistirão nos dados do problema. Especifique como será a solução. Se for um relatório, especifique o formato. Liste quaisquer suposições que você esteja fazendo sobre o problema ou as informações. Pense. Como você resolveria o problema manualmente? Desenvolva um algoritmo geral ou um plano geral de ataque.

2. Liste as tarefas principais

A listagem das tarefas principais é chamada de módulo principal. Use o português, inglês ou pseudocódigo para reformular o problema no módulo principal. Use nomes de tarefas para dividir o problema em áreas funcionais. Se o módulo principal for muito longo, você está incluindo muitos detalhes para este nível. Introduza quaisquer estruturas de controle que sejam necessárias neste ponto. Resequencie as subpartes logicamente, se necessário. Adie os detalhes para níveis inferiores.

Não se preocupe se não souber como resolver uma tarefa. Finja que tem um "amigo inteligente" que tem a resposta e deixe para pensar nisso mais tarde. Tudo o que você precisa fazer no módulo principal é dar os nomes dos módulos de nível inferior que resolvem determinadas tarefas. Use identificadores significativos.

3. Escreva os módulos restantes

Não há um número fixo de níveis. Módulos em um nível podem especificar mais módulos em níveis inferiores. Cada módulo deve ser completo, mesmo que faça referência a módulos não escritos. Faça refinamentos sucessivos em cada módulo até que cada declaração seja uma etapa concreta.

4. Resequencie e revise conforme necessário

Planeje a mudança. Não tenha medo de recomeçar. Várias tentativas e refinamentos podem ser necessários. Tente manter a clareza. Expresse-se de forma simples e direta.

A estratégia de resolução de problemas que se assemelha ao esboço de Polya é conhecida como projeto top-down (de cima para baixo). Ela produz uma hierarquia de tarefas a serem resolvidas. No Capítulo 9, apresentamos uma estratégia chamada projeto orientado a objetos, que produz uma hierarquia de objetos de dados.

? A Pedra de Roseta como sistema de tradução

A Pedra de Roseta foi descoberta pelas tropas de Napoleão em 1799. A pedra continha uma proclamação marcando o primeiro aniversário da coroação de Ptolomeu V, inscrita em três idiomas: hieróglifos, demótico (uma versão cursiva dos hieróglifos) e grego. Thomas Young, um físico britânico, e François Champollion, um egiptólogo francês, conseguiram decifrar as antigas línguas egípcias usando o grego como guia. Assim, a Pedra de Roseta forneceu a chave que desvendou a tradução dos hieróglifos egípcios.

Testando o Algoritmo

O objetivo da resolução de problemas matemáticos é produzir uma resposta específica para um problema, portanto, verificar os resultados equivale a testar o processo pelo qual a resposta foi derivada. Se a resposta estiver correta, o processo está correto. No entanto, o objetivo da resolução de problemas computacionais é criar o processo correto. O algoritmo que incorpora esse processo pode ser usado repetidamente com dados diferentes, portanto, o próprio processo deve ser testado ou verificado.

Testar um algoritmo geralmente envolve executar o programa no qual o algoritmo está codificado sob várias condições e examinar os resultados em busca de problemas. No entanto, esse tipo de teste só pode ser feito quando o programa estiver completo, ou pelo menos parcialmente completo, o que é tarde demais para confiar apenas nesse tipo de teste. Quanto mais cedo os problemas forem descobertos e corrigidos, mais barato e fácil será resolvê-los.

Claramente, precisamos realizar testes em estágios iniciais do processo de desenvolvimento. Especificamente, os algoritmos devem ser testados antes de serem implementados. Demonstramos esse processo enquanto trabalhávamos no algoritmo de mudança de base.

7.2 Algoritmos com Variáveis Simples

Variáveis simples (atômicas) são aquelas que não podem ser divididas. Elas são um valor armazenado em um lugar. Usamos variáveis simples nos algoritmos do Capítulo 6. Os números, por exemplo, são variáveis simples.

Um Algoritmo com Seleção

Suponha que você queira escrever um algoritmo para expressar qual vestimenta é apropriada para uma determinada temperatura externa. Você gostaria de usar shorts se estiver quente, mangas curtas se estiver agradável, mas não muito quente, uma jaqueta leve se a temperatura estiver fria e um casaco pesado se estiver frio. Se a temperatura estiver abaixo de zero, você fica em casa.

O módulo de nível superior (principal) apenas expressa as tarefas.

```
write "Informe a temperatura: "
read temperatura
escolha a roupa
```

As duas primeiras instruções não precisam de mais decomposição. No entanto, "escolha a roupa" precisa. Precisamos associar as temperaturas às nossas descrições. Vamos definir quente como qualquer coisa acima de 32°C, agradável como acima de 21°C, frio como acima de 10°C e congelante como acima de 0°C. Agora podemos escrever o pseudocódigo para "escolha a roupa".

```
// Escolha a roupa
if (temperatura > 32)
   write "Clima do Teresina: use shorts."
else if (temperatura > 21)
   write "Clima ideal: mangas curtas são aceitáveis."
else if (temperatura > 10)
   write "Um pouco frio: use uma jaqueta leve."
else if (temperatura > 0)
   write "Clima da Gramado: use um casaco grosso."
else
   write "Fique em casa."
```

A única maneira de chegar à segunda instrução **if** é se a primeira expressão não for verdadeira; portanto, se a segunda expressão for verdadeira, você sabe que a temperatura está entre 22°C e 32°C. Se a primeira e a segunda expressões não forem verdadeiras e a terceira for, então a temperatura está entre 11°C e 21°C. O mesmo raciocínio leva à conclusão de que o clima em Gramado está entre 0°C e 10°C, e "Fique em casa" é escrito se a temperatura for menor ou igual a 0°C. Qualquer um dos ramos pode conter uma sequência de instruções.

Algoritmos com Repetição

Existem dois tipos básicos de loops: controlados por contagem e controlados por eventos.

Loops Controlados por Contagem

Um loop controlado por contagem é um loop que repete um processo um número especificado de vezes. O mecanismo de loop simplesmente conta cada vez que o processo é repetido e testa se ele terminou antes de começar novamente. Este foi o tipo de loop que usamos no Capítulo 6.

Existem três partes distintas neste tipo de loop, que utiliza uma variável especial chamada **variável de controle** do loop. A primeira parte é a **inicialização**: a variável de controle do loop é inicializada com algum valor inicial. A segunda parte é o **teste lógico**: a variável de controle do loop atingiu um valor predeterminado? A terceira parte é o **incremento**: a variável de controle do loop é incrementada em algum valor (por exemplo, 1). O algoritmo a seguir repete um processo **limit** vezes:

```
count ← 0                  // Inicializa a variável count com 0
while (count < limit)      // Testa o valor de count
   ...                     // ... corpo do loop
   count ← count + 1       // Incrementa a variável count
...                        // Demais comandos, fora do loop
```

A variável de controle do loop, **count**, é definida como 0 fora do loop. A expressão **count < limit** é testada e o loop é executado enquanto a expressão for verdadeira. A última instrução no loop incrementa a variável de controle do loop, **count**. Quantas vezes o loop é executado? O loop é executado quando **count** é **0**, **1**, **2**, **...**, **limit – 1**. Portanto, o loop é executado **limit** vezes. O valor inicial da variável de controle do loop e o operador relacional usado na expressão booleana determinam o número de vezes que o loop é executado.

O loop **while** é chamado de loop de pré-teste (ou com teste lógico no início), porque o teste ocorre antes do loop ser executado. Se a condição for falsa inicialmente, o loop não é iniciado. O que acontece se a instrução de incremento for omitida? A expressão booleana nunca muda. Se a expressão for falsa desde o início, nada acontece; o loop simplesmente não é executado. Se a expressão for verdadeira desde o início, a expressão nunca muda, então o loop é executado para sempre, i.e., infinitamente. Na verdade, a maioria dos sistemas de computação possui um temporizador, então o programa não seria executado para sempre. Em vez disso, o programa seria interrompido com uma mensagem de erro. Um loop que nunca termina é chamado de **loop infinito**.

Este algoritmo do Capítulo 6 contém um loop controlado por contagem:

```
write "Informe a quantidade de pares: "
read numeroDePares
paresLidos ← 0
while (paresLidos < numeroDePares)
   // Corpo do loop
   ...
   paresLidos ← paresLidos + 1
```

O Pep/8 usou um ponto e vírgula para indicar que o que se seguiu foi um comentário e não parte do programa. Em nosso pseudocódigo, usamos duas barras para preceder um comentário.

Loops Controlados por Eventos

Loops nos quais o número de repetições é controlado por um evento que ocorre dentro do corpo do próprio loop são chamados de loops controlados por eventos. Ao implementar um loop controlado por eventos usando uma instrução **while**, há novamente três partes no processo: O evento deve ser **inicializado**, o evento deve ser **testado** e o evento deve ser **atualizado**. O algoritmo de conversão base do Capítulo 6 contém um laço controlado por eventos:

```
write "Informe a nova base: "
read novaBase
write "Informe o número a ser convertido: "
read numeroDecimal
resposta ← 0
quociente ← 1
while (quociente ≠ 0)
   quociente ← numeroDecimal DIV novaBase
   ...
   // Restante do corpo do loop
write "A resposta é ", resposta
```

Um loop controlado por contagem é muito simples: o processo é repetido um número especificado de vezes. A atividade em um loop controlado por evento é menos clara. Pode não ser imediatamente aparente qual deveria ser o evento.

Vejamos alguns exemplos. Primeiro, vamos ler e somar valores de dados até lermos um valor negativo. Qual é o evento? Ler um valor positivo. Como inicializamos o evento? Lemos o primeiro valor de dados. Testamos o valor para determinar se é positivo e entramos no loop se o valor for positivo. Como atualizamos o evento? Lemos o próximo valor de dados. Aqui está o algoritmo:

```
read valor                // Inicializa o evento
while (valor >= 0)        // Testa o evento
   ...                    // ... corpo do loop
   read valor             // Atualiza o evento
...                       // Demais comandos, fora do loop
```

Agora vamos escrever o algoritmo para ler e somar valores positivos até que 10 tenham sido contados. Ignoraremos valores zero ou negativos. Qual é o evento? O número de valores positivos lidos é menor que 11. Isso significa que devemos manter uma contagem do número de valores positivos à medida que os lemos; vamos chamá-lo de **posCount**. Como inicializamos o evento? Nós definimos **posCount** como 0. Testamos **posCount** em relação a 10 e saímos do loop quando **posCount** atinge 10. Como atualizamos o evento? Incrementamos **posCount** cada vez que lemos um valor positivo.

```
sum ← 0                          // Inicializa sum com zero
posCount ← 0                     // Inicializa o evento
while (posCount < 10)            // Testa o evento
   read valor
   if (valor > 0)                // A contagem deve ser atualizada?
      posCount ← posCount + 1    // Atualiza o evento
      sum ← sum + valor          // Adiciona valor mais sum dentro de sum
...                              // Demais comandos, fora do loop
```

Este não é um loop controlado por contagem porque não lemos 10 valores: lemos valores até termos lido 10.

Observe a estrutura de controle de seleção embutida no laço. As instruções a serem executadas ou puladas em qualquer estrutura de controle podem ser instruções simples ou compostas (blocos de instruções recuadas) – não há restrição quanto ao que essas instruções podem ser. Como consequência, a instrução a ser pulada ou repetida pode conter uma estrutura de controle. Instruções de seleção podem ser aninhadas em estruturas de laço; estruturas de laço podem ser aninhadas em instruções de seleção. Estruturas nas quais uma estrutura de controle está embutida em outra estrutura de controle são chamadas de estruturas aninhadas.

* Estrutura aninhada (lógica aninhada): Uma estrutura na qual uma estrutura de controle está embutida em outra.

Vamos analisar outro exemplo: encontrar a raiz quadrada de um número.

Raiz Quadrada

A maioria dos estudantes precisa calcular uma raiz quadrada na escola. Um algoritmo bastante complicado foi desenvolvido que funciona com valores inteiros. Não examinaremos esse algoritmo aqui, mas usaremos uma aproximação muito mais simples que funciona tanto com números reais quanto com números inteiros.

Dado o número cuja raiz quadrada você deseja encontrar, pegue o número e chute a resposta; em seguida, pegue o palpite e eleve-o ao quadrado. Se o seu palpite estiver correto, o palpite ao quadrado é igual ao valor original. Caso contrário, ajuste o palpite e comece de novo. Este processo continua até que o palpite ao quadrado esteja próximo o suficiente. Você entendeu o problema? Se não, releia este parágrafo.

Agora, vamos delinear as principais tarefas:

```
Leia o quadrado.
Calcule a raiz quadrada.
Escreva o quadrado e a raiz quadrada.
```

Leia o quadrado não precisa de expansão adicional. **Calcular a raiz quadrada** precisa de expansão adicional, porque é o coração do algoritmo. Claramente, há um loop: continuamos refinando a estimativa até

que ela seja boa o suficiente. É um loop controlado por contagem ou um loop controlado por eventos? Como não temos ideia de quantas iterações o processo levará, ele deve ser um loop controlado por eventos.

O que queremos dizer com "bom o suficiente"? Digamos que, se a diferença entre a estimativa ao quadrado e o valor original estiver dentro de mais ou menos 0,001, a estimativa é boa o suficiente; chamaremos essa diferença de **epsilon**. Como medimos "mais ou menos"? Tomamos o valor absoluto da diferença. Indicamos isso pela expressão **abs(epsilon)**, que significa valor absoluto.

```
// Calcule a raiz quadrada
epsilon ← 1
while (epsilon > 0.001)
   Estime uma raiz
   epsilon ← abs(quadrado – raiz * raiz)
```

Agora, a única etapa que precisa de expansão adicional é **Estimar uma raiz**. Agora precisamos fazer perguntas: Qual é a fórmula para calcular uma nova raiz? Buscamos online por "fórmula da raiz quadrada" e encontramos a resposta na Wikipédia. Substituímos o palpite antigo (**raiz**) pela média entre o palpite antigo (**raiz**) e o **quadrado** dividido pelo palpite antigo (**raiz**).

```
// Estimar uma raiz
novaRaiz ← (raiz + (quadrado/raiz)) / 2.0
```

Ao consultar a fórmula, descobrimos que havíamos esquecido algo: Qual é o palpite original? Qualquer valor positivo funcionará, mas a solução é encontrada com menos iterações se o palpite original estiver mais próximo da raiz quadrada. Uma boa aproximação para o palpite original é o quadrado dividido por 4. Não precisamos ter variáveis para o palpite (**raiz**) antigo e o novo. Podemos chamá-lo de **raiz** e simplesmente continuar alterando seu valor. Aqui está, então, o algoritmo completo:

```
read quadrado
raiz ← quadrado / 4
epsilon ← 1
while (epsilon > 0.001)
Estime uma raiz
epsilon ← abs(quadrado – raiz * raiz)
   write o quadrado e a raiz quadrada.
```

Vamos verificar este algoritmo com um valor para o qual sabemos a resposta: 81. A FIGURA 7.4 mostra o passo a passo do algoritmo. São necessárias apenas quatro iterações para obter a resposta correta com cinco casas decimais.

Uma etapa que precisa ser expandida ainda mais é chamada de etapa abstrata. Uma etapa que não precisa ser expandida é chamada de etapa concreta. A partir daqui, coloriremos as etapas abstratas de vermelho. Cada etapa abstrata deve ser expandida separadamente.

* Etapa abstrata: Uma etapa algorítmica para a qual alguns detalhes permanecem não especificados.

* Etapa concreta: Uma etapa cujos detalhes são totalmente especificados.

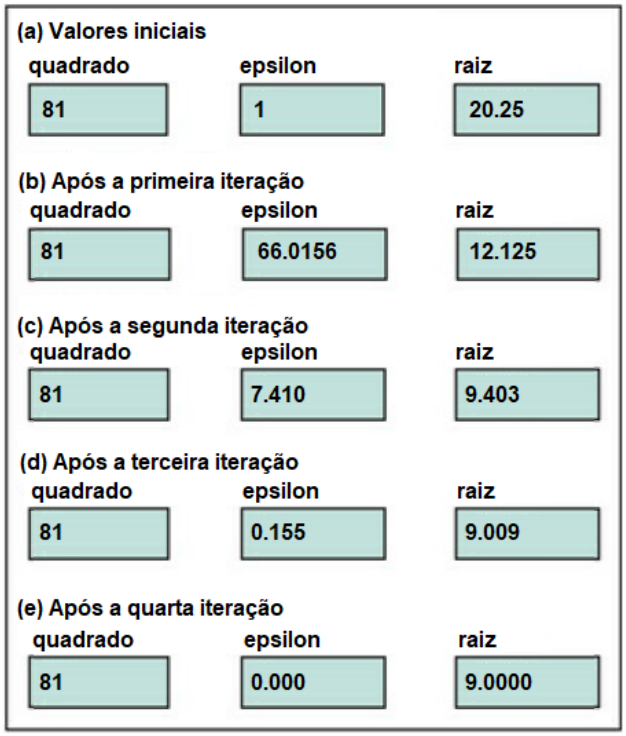


FIGURA 7.4 Passo a passo do algoritmo da raiz quadrada.

7.3 Variáveis Compostas

Os locais para armazenar valores descritos anteriormente eram todos de natureza atômica; ou seja, cada local podia conter apenas um dado, que não pode ser dividido em partes menores. Também usamos uma sequência de letras entre aspas para representar uma mensagem a ser escrita. Como você pode esperar, as letras entre aspas são chamadas de strings. Se fôssemos armazenar uma string, o número de locais necessários dependeria do número de caracteres na string.

Portanto, uma string não é atômica porque contém mais de um valor, mas tendemos a pensar em strings como atômicas de qualquer maneira, porque não acessamos as letras individualmente.

Nesta seção, descrevemos duas maneiras de coletar itens de dados juntos, dando um nome à coleção e acessando os itens individualmente ou como uma coleção.

Arrays (ou Matrizes)

Um array (ou matriz) é uma coleção nomeada de itens homogêneos na qual os itens individuais são acessados por sua posição dentro da coleção. A posição dentro da coleção é chamada de índice. Embora as pessoas geralmente comecem a contar em um, a maioria das linguagens de programação começa em zero – então é isso que faremos aqui. A FIGURA 7.5 (ao lado) mostra um array com 10 itens indexados de 0 a 9.

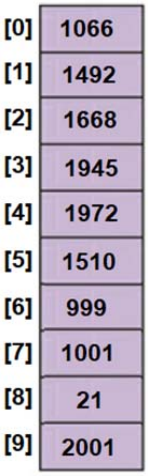


FIGURA 7.5 Um array de dez números.

Se o array for chamado de **numeros**, acessamos cada valor pela expressão **numeros[posicao]** onde **posicao**, o índice, é um valor entre 0 e 9. Aqui está o algoritmo para colocar valores nas posições de um array:

```
// Declara um array para armazenar números inteiros.
integer numeros[10]
write "Informe 10 números inteiros"
posicao ← 0
while (posicao < 10)
   read numeros[posicao]
   posicao ← posicao + 1
// ... Continua o processamento.
```

Indicamos que **numeros** é um array que pode conter valores inteiros pela declaração **integer** seguido pelo nome do array com o número de posições entre colchetes ao lado do nome. Em nossos algoritmos anteriores, não declaramos uma variável; apenas assumimos que, quando usamos um nome de variável, a variável existe. Agora que estamos usando estruturas compostas, precisamos dizer que tipo de estrutura queremos.

Algoritmos com arrays são classificados em três categorias: busca, ordenação e processamento. A busca trata-se exatamente do que diz – percorrer os itens no array, um de cada vez, procurando por um valor específico. A ordenação trata de colocar os itens no array em ordem. Se os itens forem números, eles serão colocados em ordem numérica; se forem caracteres ou strings, eles serão colocados em ordem alfabética. Um array ordenado é aquele em que os itens estão em ordem. O processamento é uma expressão genérica que abrange todos os outros cálculos feitos para ou com itens em um array.

Records (ou Registros)

Um registro é um grupo heterogêneo nomeado de itens, no qual itens individuais são acessados por nome. "Heterogêneo" significa que os elementos na coleção não precisam ser de mesmo tipo. A coleção pode conter números inteiros, valores reais, strings ou qualquer outro tipo de dado. Os registros são boas opções para agrupar itens relacionados ao mesmo objeto. Por exemplo, podemos ler um nome, uma idade e um salário por hora. Esses três itens podem então ser agrupados em um registro chamado **Empregado**. Este registro é composto por três campos: **nome**, **idade** e **salário** por hora. Podemos desenhá-los como visto na FIGURA 7.6.

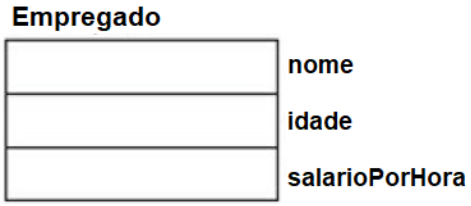


FIGURA 7.6 Registro de Funcionário.

? Software infectado por malware

Em 2013, o Centro Nacional de Coordenação de Direitos de Propriedade Intelectual (DPI), do qual o FBI é parceiro, alertou a população nos Estados Unidos sobre a possibilidade real de que softwares piratas pudessem conter malware. Esse software pode ser obtido de fornecedores desconhecidos e de redes ponto a ponto (P2P).[2]

Se declararmos uma variável de registro "empregado" do tipo "Empregado", cada campo do registro será acessado pela variável de registro, ponto e o nome do campo. Por exemplo, "**empregao.nome**" refere-se ao campo de nome da variável de registro "empregado". Não há algoritmos específicos projetados para registros, pois eles são apenas uma maneira de agrupar itens relacionados. No entanto, eles são maneiras muito úteis de se referir a um grupo de itens relacionados.

O algoritmo ao lado armazena valores nos campos do registro.

Uma terceira estrutura de dados composta, chamada classe, caracteriza a programação orientada a objetos. Discutiremos essa estrutura no Capítulo 9.

```
// Declara uma variável Empregado.
Empregado empregado
empregado.nome ← "Frank Jones"
empregado.idade ← 32
empregado.salarioPorHora ← 27.50
```

7.4 Algoritmos de Busca

Busca Sequencial

Você tem um compromisso com sua professora de inglês. Você chega ao prédio correto e procura o nome dela no diretório para encontrar o número do seu escritório. "Procurar" é sinônimo de buscar – ou seja, você pesquisa o nome dela no diretório.

Nosso primeiro algoritmo de busca segue exatamente essa definição de busca. Examinamos cada item por vez e o comparamos com aquele que estamos procurando. Se corresponder, encontramos o item. Caso contrário, examinamos o próximo item. Quando paramos? Paramos quando encontramos o item ou quando examinamos todos os itens e não encontramos uma correspondência. Isso parece um loop com duas condições finais. Vamos escrever o algoritmo usando o array **numeros**.

```
posicao ← 0
achou ← FALSE
while (posicao < 10 and achou == FALSE)
   if numeros[posicao] == chave)
      achou ← TRUE
   else
      posicao ← posicao + 1
```

Como temos uma condição composta na expressão **while**, precisamos falar um pouco mais sobre variáveis booleanas. O **and** é um operador booleano. Os operadores booleanos incluem os operadores especiais **and**, **or** e **not**. O operador **and** retorna TRUE se ambas as expressões forem TRUE e FALSE caso contrário. O operador **or** retorna FALSE se ambas as expressões forem FALSE e TRUE caso contrário. O operador **not** inverte o valor da expressão. Essas operações são consistentes com a funcionalidade das portas lógicas descritas no Capítulo 4. Nesse nível, estávamos nos referindo ao fluxo de eletricidade e à representação de bits individuais. Nesse nível, a lógica é a mesma, mas podemos falar em termos de uma expressão como sendo verdadeira ou falsa.

Podemos simplificar a segunda expressão booleana (**achou == FALSE**) usando o operador **not**. Ou seja, **not achou**, que será verdadeiro quando **achou** é falso. Então, podemos dizer

while (posicao < 10 and not achou)

Assim, o loop se repetirá enquanto o índice for menor que 10 e não tivermos encontrado o item correspondente.

Busca Sequencial em um Array Ordenado

Se soubermos que os itens no array estão ordenados, podemos parar de procurar quando passarmos o local onde o item estaria se estivesse presente no array. Ao analisarmos este algoritmo, vamos generalizar um pouco nossa busca. Em vez de sermos específicos sobre o número de itens no array, usamos uma variável de comprimento para nos dizer quantos itens válidos existem no array. O comprimento pode ser menor que o tamanho máximo, que é o número total de posições no array. À medida que os dados são lidos no array, um contador é atualizado para que sempre saibamos quantos itens de dados foram armazenados. Se o array for chamado de **dados**, os dados com os quais estamos trabalhando vão de **dados[0]** a **dados[tamanho – 1]**. As FIGURAS 7.7 e 7.8 mostram um array não ordenado e um array ordenado, respectivamente.

No array ordenado, se estivermos procurando por 76, sabemos que ele não está no array assim que examinamos os **dados[3]**, porque esta posição é onde o número estaria se estivesse lá. Aqui está o algoritmo para busca em um array ordenado incorporado em um programa completo. Usamos o índice da variável em vez da posição neste algoritmo. Os programadores frequentemente usam o índice do identificador matemático em vez do identificador intuitivo posição ou lugar ao trabalhar com arrays.

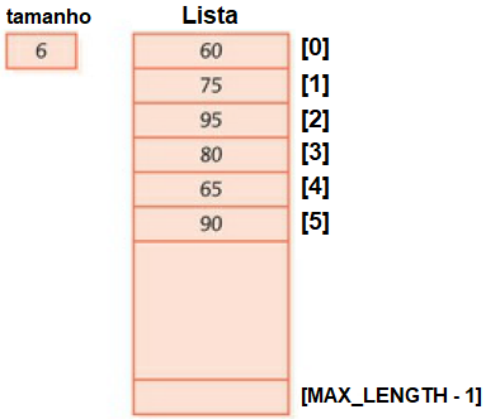


FIGURA 7.7 Um array não ordenado.

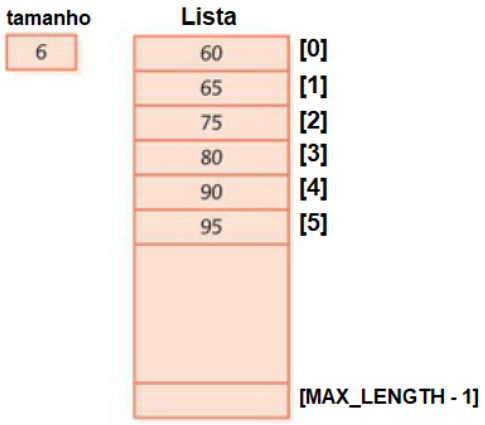


FIGURA 7.8 Um array ordenado.

```
carregar o array de valores
write "Chave de busca: "
read chave
achou ← TRUE se a chave for encontrada
if (achou)
   write "Chave encontrada"
else
   write "Chave não encontrada"
```

```
carregar o array de valores
write "Quantidade de valores: "
read tamanho
indice ← 0
while (indice < tamanho)
   read dados[indice]
   indice ← indice + 1
```

```
achou ← TRUE se a chave for encontrada
indice ← 0
achou ← FALSE
while (indice < tamanho and not achou)
   if (dados[indice] == chave)
      achou ← TRUE
   else if (dados[indice] > chave)
       indice ← tamanho
   else
       indice ← indice + 1
```

Busca Binária

Como você procuraria uma palavra em um dicionário? Certamente esperamos que você não comece na primeira página e busque a sua palavra sequencialmente! Uma busca sequencial de um array começa no início do array e continua até que o item seja encontrado ou todo o array tenha sido pesquisado sem encontrar o item.

Uma busca binária procura um item em um array usando uma estratégia diferente: dividir e conquistar. Essa abordagem imita o que você provavelmente faz ao procurar uma palavra no dicionário. Você começa no meio e então decide se a sua palavra está na seção à direita ou à esquerda seção. Em seguida, você procura na seção correta e repete a estratégia.

O algoritmo de busca binária assume que os itens no array pesquisado estão ordenados e ele encontra o item ou elimina metade do array com uma comparação. Em vez de procurar o item começando no início do array e avançando sequencialmente, o algoritmo começa no meio do array em uma busca binária. Se o item que estamos buscando for menor que o item do meio, sabemos que o item não estará na segunda metade do array. Em seguida, continuamos buscando os dados na primeira metade do array. Veja a FIGURA 7.9 (ao lado).

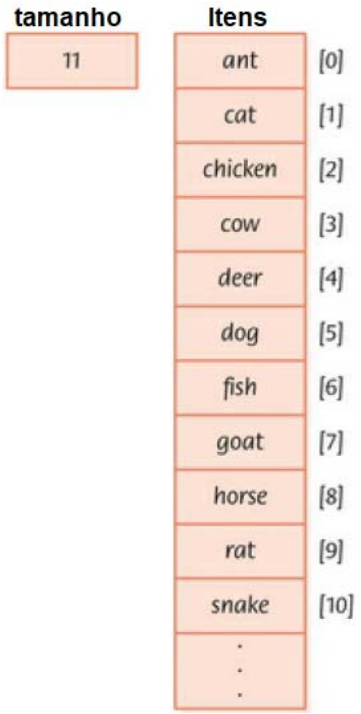


FIGURA 7.9 Exemplo de busca binária.

* Busca binária: busca por um item em uma lista já ordenada, eliminando grandes porções dos dados em cada comparação.

Mais uma vez, examinamos o elemento "do meio" (que na verdade é o item a 25% do caminho para dentro do array). Se o item que estamos buscando for maior que o item do meio, continuamos buscando entre o meio e o fim do array. Se o item do meio for igual ao que você está buscando, a busca termina. O processo continua dessa maneira, com cada comparação cortando pela metade a parte do array onde o item pode estar. Ela termina quando o item é encontrado ou quando a parte do array onde o item pode estar fica vazia.

```
Busca binária booleana
esq ← 0
dir ← tamanho – 1
achou ← FALSE
while (esq <= dir and achou)
   meio ← (esq + dir) / 2
   if (chave == dados[meio])
      achou ← TRUE
   else if (chave < dados[indice])
       esq ← meio - 1
   else
       dir ← meio + 1
return achou
```

Vamos verificar (analisar) o algoritmo, buscando por *cat*, *fish* e *zebra*. Em vez de caixas, usamos uma forma tabular na FIGURA 7.10 para economizar espaço.

O algoritmo de busca binária é realmente mais rápido que o algoritmo de busca sequencial? A TABELA 7.1 mostra o número médio de comparações necessárias para encontrar um item usando uma busca sequencial e uma busca binária. Se a busca binária é tão mais rápida, por que não a usamos sempre? Mais cálculos são necessários para cada comparação em uma busca binária porque precisamos calcular o índice do meio. Além disso, a matriz já deve estar ordenada. Se a matriz já estiver ordenada e o número de itens for maior que 20, um algoritmo de busca binária é a melhor escolha.

Buscando por "cat"

Esquerda	Direita	Meio	Comparação	
0	10	5	cat < dog	
0	4	2	cat < chicken	
0	1	0	cat > ant	
1	1	1	cat = cat	**return TRUE**

Buscando por "fish"

Esquerda	Direita	Meio	Comparação	
0	10	5	fish > dog	
6	10	8	fish < horse	
6	7	6	fish = fish	**return TRUE**

Buscando por "zebra"

Esquerda	Direita	Meio	Comparação	
0	10	5	zebra > dog	
6	10	8	zebra > horse	
9	10	9	zebra > rat	
10	10	10	zebra > snake	
11	10		first > last	**return FALSE**

FIGURA 7.10 Traços da busca binária.

TABELA 7.1 Número Médio de Comparações.

Tamanho	Busca Sequencial	Busca Binária
10	5.5	2.9
100	50.5	5.8
1000	500.5	9.0
10000	5000.5	12.0

7.5 Ordenação

Todos sabemos o que é ordenação. Ordenamos nossa playlist de músicas, nossas estantes de livros e até mesmo nossas prioridades. Ordenar é colocar as coisas em ordem. Na computação, transformar um array não ordenado em um array ordenado é uma operação comum e útil. Livros inteiros já foram escritos sobre algoritmos de ordenação. O objetivo é criar ordenações melhores e mais eficientes. Como ordenar um grande número de elementos pode ser extremamente demorado, um bom algoritmo de ordenação é considerado altamente desejável. Na verdade, esta é uma área em que os programadores às vezes são incentivados a sacrificar a clareza em favor da velocidade de execução.

Nesta seção, apresentamos diversos algoritmos de ordenação para lhe dar uma ideia de quantas maneiras diferentes existem para resolver o mesmo problema.

Ordenação por Seleção

Se entregássemos a você uma pilha de cartões com nomes de pessoas e pedíssemos que as colocasse em ordem por nome, você provavelmente percorreria as fichas procurando o nome que vem primeiro no alfabeto. Você então colocaria essa carta como a primeira em uma nova pilha de cartas. Como você determinaria qual carta vem primeiro? Você poderia virar a primeira carta de lado para se lembrar dela.

Se você encontrar uma carta que venha antes da carta virada, você pode virar a primeira carta de volta e virar a nova "primeira" carta para se lembrar dela. Depois de reexaminar todas as cartas, a que estiver virada é a primeira. Você a retira para iniciar o baralho de cartas ordenado. Este processo continuará até que todas as cartas tenham sido movidas para a nova pilha.

```
while tiver cartas na pilha da mesa
   encontre a menor carta na primeira pilha da mesa
   mova a menor carta para a nova pilha
```

O algoritmo de ordenação por seleção é provavelmente o mais fácil porque espelha como classificaríamos uma lista de valores se tivéssemos que fazê-lo manualmente. Nosso baralho de cartas é um array de nomes. O novo baralho é um array de nomes ordenados. Removemos itens do primeiro e os colocamos em lugares sucessivos no segundo. Lembramos do menor até agora salvando sua posição em uma variável temporária.

Este algoritmo é simples, mas tem uma desvantagem: requer espaço para dois baralhos completos (arrays). Embora não tenhamos falado sobre considerações de espaço de memória, essa duplicação é claramente um desperdício. Um pequeno ajuste nessa abordagem manual, no entanto, elimina a necessidade de duplicar espaço. À medida que você move o menor item para o novo array, um espaço livre é aberto onde ele costumava estar. Em vez de escrever esse nome em uma segunda lista, podemos trocá-lo pelo nome atualmente na posição onde o novo nome deve ficar. Essa "lista manual" é representada em um array.

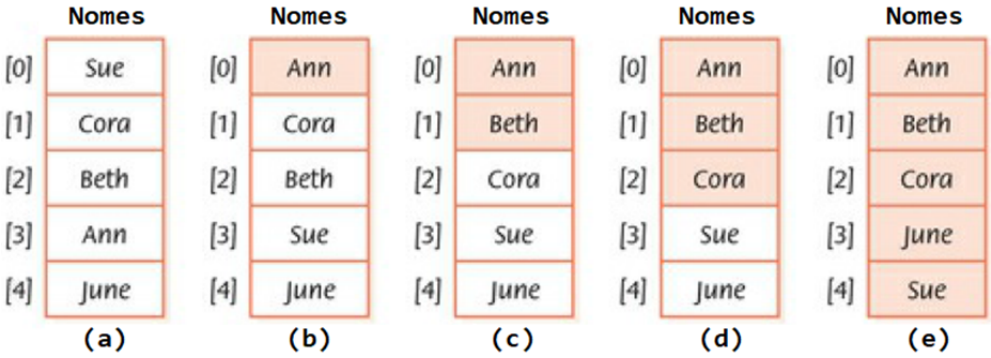


FIGURA 7.11 Exemplos de ordenação por seleção (os elementos ordenados são sombreados).

Vejamos um exemplo: a ordenação do array de cinco elementos mostrado na FIGURA 7.11. Devido à simplicidade desse algoritmo, ele geralmente é o primeiro método de ordenação que os alunos aprendem.

Vamos visualizar o array como contendo duas partes: a parte não ordenada (não sombreada) e a parte ordenada (sombreada). Cada vez que adicionamos um item ao seu devido lugar, estamos reduzindo a parte não ordenada e estendendo a parte ordenada. A ordenação começa com todos os itens na parte não ordenada e termina com todos os itens na parte ordenada. Segue o algoritmo escrito para espelhar essa visualização.

```
// Ordenação por Seleção (Selection Sort)
primeiroNaoOrdenado ← 0
while (nenhum ordenado)
   encontre o menor item não ordenado
   troque primeiroNaoOrdenado pelo menor
   primeiroNaoOrdenado ← primeiroNaoOrdenado + 1
```

Este algoritmo inclui apenas três etapas abstratas (coloridas em vermelho): determinar quando o array é ordenado, encontrar o índice do menor elemento e trocar o conteúdo de duas posições. Ao passar da Figura 7.11(d) para a 7.11(e), adicionamos os dois últimos itens à parte sombreada do array. Isso sempre acontece porque, quando o menor dos dois últimos itens é colocado em seu devido lugar, o último item também está em seu devido lugar. Assim, o loop continua enquanto **primeiroNãoOrdenado** for menor que o comprimento do array – 1.

```
// Nenhum ordenado
primeiroNaoOrdenado < tamanho - 1
```

Como você encontraria o nome que vem primeiro no alfabeto na parte não ordenada da lista se estivesse ordenando-a manualmente? Você vê (e registra mentalmente) o primeiro e então percorre a lista (virando os cartões) até encontrar um que venha antes do primeiro. Você se lembra deste menor e continua examinando a lista em busca de um nome que venha antes deste no alfabeto. O processo de lembrar o menor até o momento até encontrar um menor é repetido até chegar ao final da lista. Este algoritmo manual é exatamente o que usamos aqui, exceto que devemos lembrar o índice do menor item porque o trocaremos com o item na posição **primeiroNaoOrdenado**. Em termos da nossa lista, procuramos o menor item na parte não ordenada, que vai de **primeiroNaoOrdenado** até **tamanho – 1**.

```
// Encontre o menor item não ordenado
indiceDoMenor ← primeiroNaoOrdenado
indice ← primeiroNaoOrdenado + 1
while (indice <= tamanho - 1)
   if (dados[indice] < dados[indiceDoMenor])
      indiceDoMenor ← indice
   indice ← indice + 1
```

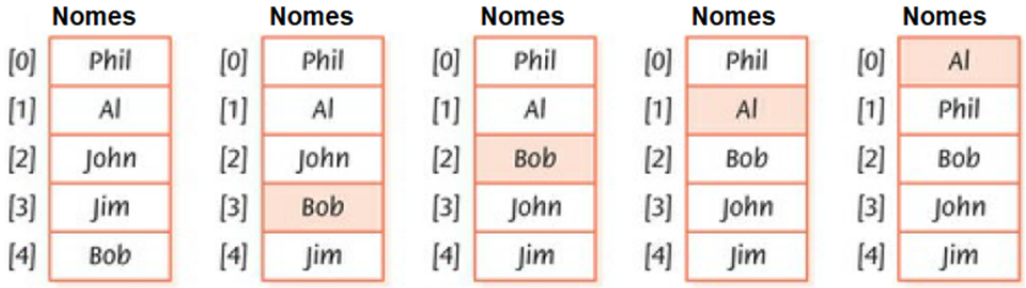
Quantos copos são necessários para trocar o conteúdo de dois copos? Três. Você precisa de um copo para armazenar temporariamente o conteúdo de um copo enquanto despeja o conteúdo do outro copo no primeiro. Trocar o conteúdo de dois lugares na memória é exatamente o mesmo problema. O algoritmo de troca deve ter os índices dos dois itens a serem trocados.

```
// Troque primeiroNaoOrdenado pelo menor
itemTemporario ← dados[primeiroNaoOrdenado]
dados[primeiroNaoOrdenado] ← dados[indeceDoMenor]
dados[indeceDoMenor] ← itemTemporario
```

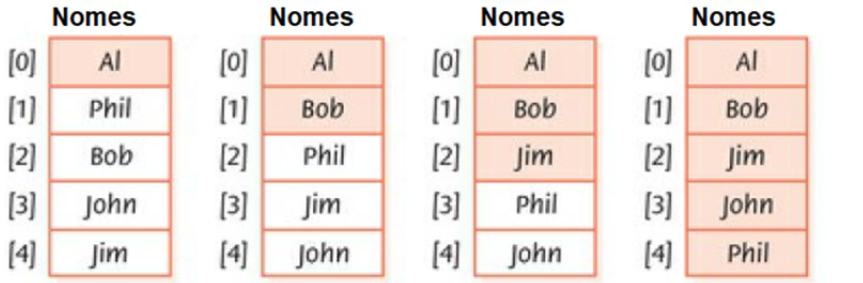
Ordenação por Bolha

A ordenação por bolha é uma ordenação por seleção que usa um esquema diferente para encontrar o valor mínimo. Começando com o último elemento do array, comparamos pares sucessivos de elementos, trocando-os sempre que o elemento inferior do par for menor que o superior. Vide FIGURA 7.12(a). Dessa forma, o menor elemento "sobe como uma bolha" para o topo do array. Cada iteração coloca o menor item não classificado em seu lugar correto usando a mesma técnica, mas também altera as posições dos outros elementos no array. Vide FIGURA 7.12(b).

Antes de escrevermos este algoritmo, devemos fazer uma observação: a ordenação por bolha é um algoritmo de ordenação muito lento. Os algoritmos de ordenação geralmente são comparados com base no número de iterações necessárias para ordenar um array, e essa abordagem requer uma iteração para cada item no array, exceto o último. Além disso, durante cada algoritmo, muitas trocas ocorrem. Por que, então, nos preocupamos em mencionar a ordenação por bolha se ela é tão ineficiente? Porque uma pequena alteração no algoritmo de ordenação a torna uma excelente escolha para uso em determinadas circunstâncias. Vamos aplicar o algoritmo a um array já ordenado. Veja a coluna mais à direita na Figura 7.12(b).



(a) Primeira iteração (elementos ordenados estão sombreados)



(b) Iterações restantes (elementos ordenados estão sombreados)

FIGURA 7.12 Ordenação por bolha.

Comparamos Phil com John e não os trocamos. Comparamos John com Jim e não os trocamos. Comparamos Jim com Bob e não os trocamos. Comparamos Bob com Al e não os trocamos. Se nenhum valor for trocado durante uma iteração, o array é ordenado. Definimos uma variável booleana como FALSE antes de entrar no loop e a definimos como TRUE se ocorrer uma troca. Se a variável booleana ainda for FALSE, o array é ordenado.

Compare o processamento da ordenação por bolha com a ordenação por seleção em um array já ordenado. O algoritmo de ordenação por seleção não nos dá como determinar se o array está ordenado; portanto, analisaremos todo o algoritmo.

```
// Ordenação por Bolhas (Bubble Sort)
primeiroNaoOrdenado ← 0
trocou ← TRUE
while (primeiroNaoOrdenado < tamanho – 1 and trocou)
   trocou ← FALSE
   Borbulhar o menor item da parte não ordenada
   primeiroNaoOrdenado ← primeiroNaoOrdenado + 1
```

```
// Borbulhar o menor item da parte não ordenada
indice ← tamanho - 1
while (indice > primeiroNaoOrdenado)
   if (dados[indice] < dados[indice – 1])
      Trocar dados[indice] com dados[indice – 1]
      trocou ← TRUE
   indice ← indice + 1
```

? O que é piggybacking?

Usar o serviço de acesso à Internet sem fio de outro assinante sem a permissão ou conhecimento explícito do assinante é chamado de piggybacking. É ético? Alguns comparam isso a ler o jornal por cima do ombro da pessoa à sua frente enquanto estão em um trem. Outros comparam isso a entrar em uma casa só porque a porta está destrancada. É legal? Algumas jurisdições permitem, outras proíbem e outras não são bem definidas. O que você acha?

Ordenação por Inserção

Se você tiver apenas um item no array, ele será ordenado. Se você tiver dois itens, poderá compará-los e trocá-los, se necessário. Agora, os dois primeiros estão ordenados em relação a si mesmos. Pegue o terceiro

item e coloque-o em seu lugar em relação aos dois primeiros. Agora, os três primeiros itens estão ordenados em relação um ao outro. O item que está sendo adicionado à parte ordenada pode ser empilhado como na ordenação por bolha. Quando você encontrar um lugar onde o item que está sendo inserido é menor que o item no array, você o armazena lá. No algoritmo **corrente** é o item que está sendo inserido na parte ordenada. Veja a FIGURA 7.13.

```
// Ordenação por Inserção (Insertion Sort)
corrente ← 1
while (corrente < tamanho)
   indice ← corrente
   localEncontrado ← FALSE
   while (indice > 0 and not localEncontrado)
      if (dados[indice] < dados[indice – 1])
         Trocar dados[indice] com dados[indice – 1]
         indice ← indice - 1
      else
         localEncontrado ← TRUE
   corrente ← corrente + 1
```

A cada iteração de uma ordenação por seleção, mais um item é colocado em seu lugar permanente. A cada iteração de uma ordenação por inserção, mais um item é colocado em seu lugar apropriado em relação aos itens acima dele.

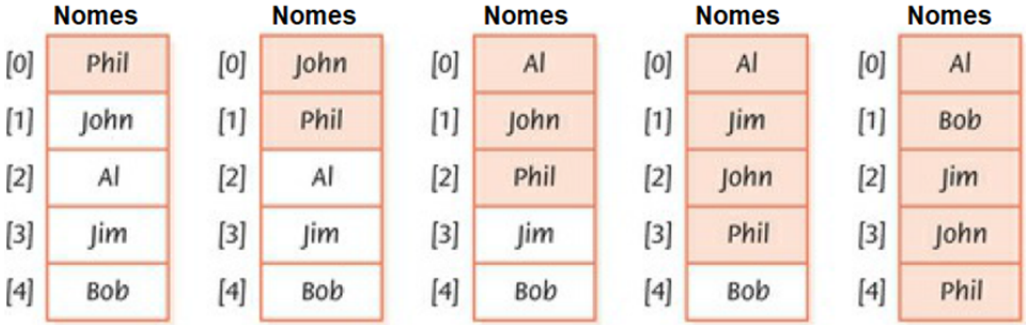


FIGURA 7.13 Ordenação por inserção.

7.6 Algoritmos Recursivos

Quando um algoritmo usa seu próprio nome dentro de si mesmo, ele é chamado de algoritmo recursivo. Ou seja, se um nome de tarefa em um nível chama a si mesmo, a chamada é conhecida como chamada recursiva. Recursão – a capacidade de um algoritmo chamar a si mesmo – é uma estrutura de controle alternativa à repetição (loop). Em vez de usar uma instrução de repetição para executar um segmento do algoritmo, tal algoritmo usa uma instrução de seleção para determinar se deve repetir o algoritmo chamando-o novamente ou interromper o processo.

* Recursão: A capacidade de um algoritmo de chamar a si mesmo.

Cada algoritmo recursivo possui pelo menos dois casos: o **caso base** e o **caso geral**. O caso base é aquele para o qual temos uma resposta; o caso geral expressa a solução em termos de uma chamada a si mesmo com uma versão menor do problema. Como o caso geral resolve uma versão cada vez menor do problema original, eventualmente o programa alcança o caso base, onde uma resposta é conhecida. Nesse ponto, a recursão termina.

Associada a cada problema recursivo está alguma medida do tamanho do problema. O tamanho deve se tornar menor a cada chamada recursiva. O primeiro passo em qualquer solução recursiva é, portanto, determinar o **fator de tamanho**. Se o problema envolver um valor numérico, o fator de tamanho pode ser o próprio valor. Se o problema envolver uma estrutura, o fator de tamanho é provavelmente o tamanho da estrutura.

Até agora, demos um nome a uma tarefa em um nível e expandimos a tarefa em um nível inferior. Em seguida, reunimos todas as peças no algoritmo final. Com algoritmos recursivos, precisamos ser capazes de fornecer ao algoritmo valores de dados diferentes a cada vez que o executamos. Portanto, antes de prosseguirmos com a recursão, precisamos examinar uma nova estrutura de controle: a **instrução**

subprogram. Embora ainda estejamos no nível do algoritmo, essa estrutura de controle utiliza a palavra **subprogram**.

Instruções de Subprograma

Podemos dar um nome a uma seção de código e, em seguida, usá-lo como uma instrução em outra parte do programa. Quando o nome é encontrado, o processamento na outra parte do programa é interrompido enquanto o código nomeado é executado. Quando o código nomeado termina a execução, o processamento é retomado com a instrução logo abaixo de onde o nome ocorreu. O local onde o nome do código aparece é conhecido como **unidade de chamada**.

Existem duas formas básicas de subprogramas: código nomeado que executa uma tarefa específica (subprogramas void) e código nomeado que também executa uma tarefa, mas retorna um único valor para a unidade de chamada (subprogramas que retornam valor). A primeira forma é usada como uma instrução na unidade de chamada; a segunda forma é usada em uma expressão na unidade de chamada, onde o valor retornado é então usado na avaliação da expressão.

Subprogramas são ferramentas poderosas para abstração. A listagem de um subprograma nomeado permite que o leitor do programa veja que uma tarefa está sendo executada sem precisar se preocupar com os detalhes da implementação da tarefa. Se um subprograma precisar de informações para executar sua tarefa, colocamos o nome do valor dos dados entre parênteses, ao lado do cabeçalho do subprograma. Se o subprograma retornar um valor para a unidade chamadora, ele usa a palavra **return** seguida do nome dos dados a serem retornados. Veja a FIGURA 7.14.

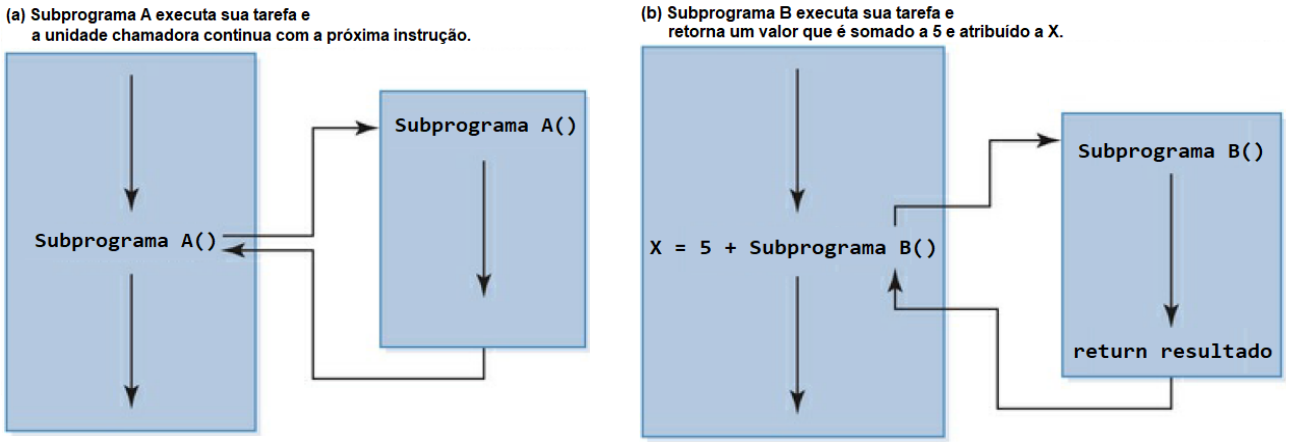


FIGURA 7.14 Fluxo de controle do subprograma.

Fatorial Recursivo

O fatorial de um número é definido como o número multiplicado pelo produto de todos os números entre ele mesmo e 0: **`N! = N * (N – 1)!`**

O fatorial de 0 é 1. O fator de tamanho é o número para o qual estamos calculando o fatorial. Temos um caso base: **`Fatorial(0) = 1`**

Também temos um caso geral: **`Fatorial(N) = N * Fatorial(N – 1)`**

Uma instrução **if** pode avaliar **N** para verificar se é 0 (o caso base) ou maior que 0 (o caso geral). Como **N** está claramente diminuindo a cada chamada, o caso base é eventualmente alcançado.

```
write "Informe N: "
read N
resultado ← Fatorial(N)
   write N + "! = " + resultado
```

```
// Fatorial(N)
if (N == 0)
   return 1
else
   return N * Fatorial(N-1)
```

Cada vez que **Fatorial()** é chamado, o valor de **N** fica menor. Os dados fornecidos a cada vez são chamados de *argumento*. O que acontece se o argumento for um número negativo? O subprograma continua chamando a si mesmo até que o sistema de suporte em tempo de execução fique sem memória. Essa situação, chamada de *recursão infinita*, é equivalente a um loop infinito.

Busca Binária Recursiva

Embora tenhamos codificado a busca binária usando um loop, o algoritmo de busca binária parece recursivo. Paramos quando encontramos o item ou quando sabemos que ele não está lá (casos base). Continuamos a procurar o item na seção do array onde ele pode estar, se estiver presente. Um algoritmo recursivo deve ser chamado a partir de um algoritmo não recursivo, como fizemos com o algoritmo fatorial. Aqui, o subprograma precisa saber o índice da **esquerda** e o índice da **direita** dentro dos quais está pesquisando. Em vez de redefinir o esquerda ou o direita como fizemos na versão iterativa, simplesmente chamamos o algoritmo novamente com os novos valores para o índice **esquerda** e o índice **direita**.

```
// Ordenação por Inserção (Insertion Sort)
BuscaBinaria(esq, dir)
   if (esq > dir)
     return FALSE
   else
     meio ← (esq + dir) / 2
     if (item == dados[meio])
        return TRUE
     else if (item < dados[meio])
        BuscaBinaria(esq, meio-1)
     else
        BuscaBinaria(meio+1, dir)
```

? Platão era um designer de software?

A filosofia passou 2500 anos trabalhando em conceitos que agora são parte integrante do design de software. Por exemplo, os universais e particulares de Platão podem ser vistos como classes e instâncias de classes na programação orientada a objetos.[3]

Quicksort

O algoritmo Quicksort, desenvolvido por C. A. R. Hoare, baseia-se na ideia de que é mais rápido e fácil ordenar duas listas pequenas do que uma maior. O nome vem do fato de que, em geral, o Quicksort consegue ordenar uma lista de elementos de dados com bastante rapidez. A estratégia básica dessa ordenação é "dividir para conquistar".

Se você recebesse uma grande pilha de provas finais para classificar por nome, você poderia usar a seguinte abordagem: Escolha um valor de divisão, digamos L, e divida a pilha de provas em duas pilhas, A–L e M–Z. (Observe que as duas pilhas não contêm necessariamente o mesmo número de provas.) Em seguida, pegue a primeira pilha e subdivida-a em duas pilhas, A–F e G–L. A pilha A–F pode ser subdividida em A–C e D–F. Esse processo de divisão continua até que as pilhas sejam pequenas o suficiente para serem facilmente ordenadas manualmente. O mesmo processo é aplicado à pilha M–Z. Eventualmente, todas as pilhas pequenas e classificadas podem ser empilhadas uma sobre a outra para produzir um conjunto classificado de provas. Veja a FIGURA 7.15.

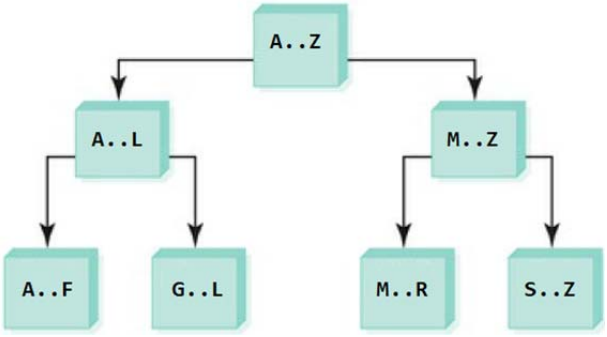


FIGURA 7.15 Ordenando uma lista usando o algoritmo Quicksort.

Esta estratégia é baseada em recursão – a cada tentativa de ordenar a pilha de testes, a pilha é dividida e, em seguida, a mesma abordagem é usada para ordenar cada uma das pilhas menores (um caso menor). Este processo continua até que as pilhas pequenas não precisem ser mais divididas (o caso base). As variáveis **first** e **last** no algoritmo Quicksort refletem a parte dos **dados** do array que está sendo processada no momento.

```
// Ordenação Rápida (Quicksort)
Quicksort(first, last)
   if (first < last)                        // Há mais de um item
      Selecionar valorDivisao
      Dividir(valorDivisao)
                                            // Array entre first e
                                            // splitPoint-1 <= splitVal
                                            // data[splitPoint] == splitVal
                                            // Array entre splitPoint + 1
                                            // e last > splitVal
      Quicksort(first, pontoDivisao - 1)
      Quicksort(pontoDivisao + 1, last)
```

Como selecionamos **valorDivisao**? Uma solução simples é usar qualquer valor que esteja em **data[first]** como o valor de divisão. Vejamos um exemplo usando **data[first]** como **valorDivisao**.

valorDivisao = 9

9	20	6	10	14	8	60	11
[first]							[last]

Após a chamada para **Dividir()**, todos os itens menores ou iguais a **valorDivisao** ficam no lado esquerdo do array e todos os itens maiores que **valorDivisao** ficam no lado direito do array.

FIGURA

Valores menores			Valores maiores				
9	8	6	10	14	20	60	11
[first]		[pontoDivisao]					[last]

As duas "metades" se encontram em **pontoDivisao**, o índice do último item menor ou igual a **valorDivisao**. Observe que não sabemos o valor de **pontoDivisao** até que o processo de divisão esteja concluído. Podemos então trocar **valorDivisao(dados[first])** pelo valor em **dados[pontoDivisao]**.

Valores menores			Valores maiores				
6	8	9	10	14	20	60	11
[first]		[pontoDivisao]					[last]

Nossas chamadas recursivas para **Quicksort()** usam esse índice (**pontoDivisao**) para reduzir o tamanho do problema no caso geral.

A chamada **Quicksort(first, pontoDivisao – 1)** ordena a "metade" esquerda do array. A chamada **Quicksort(pontoDivisao + 1, last)** ordena a "metade" direita do array. (As "metades" não têm necessariamente o mesmo tamanho.) A variável **valorDivisao** já está em sua posição correta em **dados[pontoDivisao]**.

Qual é o caso base? Quando o segmento examinado possui apenas um item, não precisamos continuar. Isso é representado no algoritmo pela ausência da instrução **else**. Se houver apenas um valor no segmento que está sendo ordenado, ele já está em seu lugar.

Precisamos encontrar uma maneira de obter todos os elementos iguais ou menores que **valorDivisao** de um lado de **valorDivisao** e todos os elementos maiores que **valorDivisao** do outro lado. Fazemos isso movendo um par de índices das extremidades em direção ao meio do array, procurando por itens que estejam do lado errado do valor de divisão. Quando encontramos pares que estejam do lado errado, trocamos os valores e continuamos trabalhando até o meio do array.

```
// Dividir(valorDivisao)
Dividir(valorDivisao)
   left ← first + 1
   right ← last
   while (left <= right)
      Incremente(left) até dados[left] > valorDivisao ou left > right
      Decremente(right) até dados[right] < valorDivisao ou left > right
      if (left < right)
         Troque dados[left] com dados[right]
   pontoDivisao ← right
   Troque dados[first] com dados[pontoDivisao]
   return pontoDivisao
```

Embora ainda tenhamos uma etapa abstrata, podemos parar, pois já expandimos essa etapa abstrata em um problema anterior. Isso levanta um ponto muito importante: nunca reinvente a roda. Uma etapa abstrata em um algoritmo pode ter sido resolvida anteriormente por você ou por outra pessoa. A FIGURA 7.16 mostra um exemplo desse algoritmo de divisão.

O Quicksort é uma excelente abordagem de ordenação se os dados a serem ordenados estiverem em ordem aleatória. Se os dados já estiverem ordenados, no entanto, o algoritmo degenera para que cada divisão tenha apenas um item.

A recursão é uma ferramenta muito poderosa e elegante. No entanto, nem todos os problemas podem ser facilmente resolvidos recursivamente, e nem todos os problemas que têm uma solução recursiva óbvia devem ser resolvidos recursivamente. Mesmo assim, uma solução recursiva é preferível para muitos problemas. Se a declaração do problema se enquadra logicamente em dois casos (um caso base e um caso geral), a recursão é uma alternativa viável.

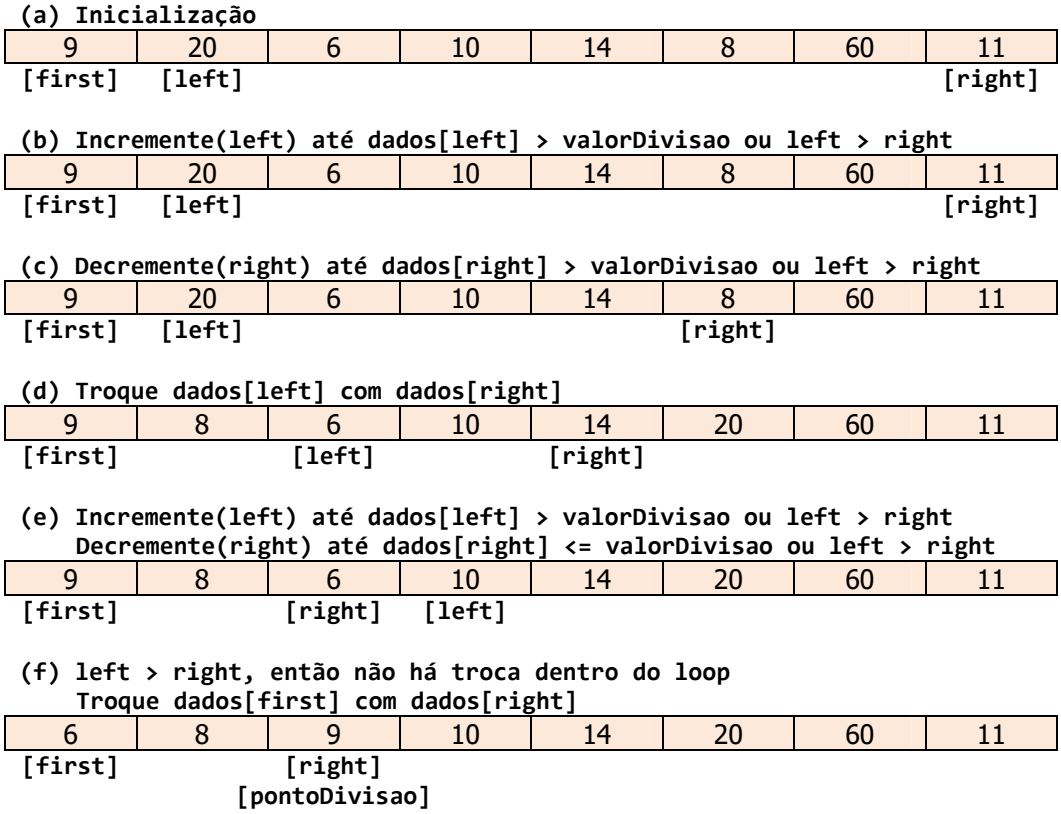


FIGURA 7.16 Algoritmo de divisão.

7.7 Threads Importantes

Nos dois últimos capítulos, mencionamos vários tópicos de passagem que são importantes não apenas na resolução de problemas, mas também na computação em geral. Vamos revisar alguns dos tópicos comuns discutidos nestes capítulos.

Ocultação de Informações

Mencionamos a ideia de adiar os detalhes diversas vezes. Usamos isso no contexto de dar um nome a uma tarefa e não nos preocuparmos com como a tarefa será implementada para mais tarde. Adiar os detalhes em um projeto tem vantagens distintas. O projetista vê apenas os detalhes que são relevantes em um nível específico do projeto. Essa prática, chamada ocultação de informações, torna os detalhes em um nível inferior inacessíveis durante o projeto dos níveis superiores.

* Ocultação de Informações: A prática de ocultar os detalhes de um módulo com o objetivo de controlar o acesso aos detalhes do módulo.

Essa prática deve parecer muito estranha! Por que os detalhes não deveriam ser acessíveis enquanto o algoritmo está sendo projetado? O designer não deveria saber tudo? Não. Se o designer conhece os

detalhes de baixo nível de um módulo, é mais provável que ele baseie o algoritmo do módulo nesses detalhes – e são precisamente esses detalhes de baixo nível que têm maior probabilidade de mudar. Se mudarem, todo o módulo terá que ser reescrito.

Abstração

Abstração e ocultação de informação são dois lados da mesma moeda. A ocultação de informação é a prática de ocultar detalhes; abstração é o resultado com os detalhes ocultos. Como dissemos no Capítulo 1, uma abstração é um modelo de um sistema complexo que inclui apenas os detalhes essenciais para o observador. Tomemos, por exemplo, Daisy, a spaniel inglesa. Para seu dono, ela é um animal de estimação; para um caçador, ela é um cão de caça; e para o veterinário, ela é um mamífero. Seu dono observa o rabo abanando de Daisy, ouve seu latido quando ela é deixada do lado de fora e vê os pelos que ela deixa por toda parte. O caçador vê uma ajudante bem treinada que conhece seu trabalho e o faz bem. O veterinário vê todos os órgãos, carne e ossos que a compõem. Veja a FIGURA 7.17.

* Abstração Um modelo de um sistema complexo que inclui apenas os detalhes essenciais para o observador.



FIGURA 7.17 Diferentes visões do mesmo conceito.

Em computação, um algoritmo é uma abstração das etapas necessárias para implementá-lo. O usuário casual de um programa que inclui o algoritmo, que vê apenas a descrição de como executar o programa, é como o dono do cachorro: ele ou ela vê apenas a superfície. O programador, que incorpora o algoritmo de outra pessoa em seu programa, é como o caçador que usa o cachorro bem treinado: ele ou ela usa o algoritmo para um propósito. O implementador do algoritmo, que deve entendê-lo completamente, é como o veterinário: ele ou ela deve ver o funcionamento interno para implementar o algoritmo.

Em computação, vários tipos de abstração são aparentes. Abstração de dados refere-se à visão dos dados; é a separação da visão lógica dos dados de sua implementação. Por exemplo, o computador do seu banco pode representar números em complemento de dois ou em complemento de um, mas essa distinção não lhe interessa, desde que seus extratos bancários estejam corretos.

Abstração procedural refere-se à visão das ações; é a separação da visão lógica de uma ação de sua implementação. Por exemplo, quando demos um nome a um subprograma, estávamos praticando abstração procedural.

Um terceiro tipo de abstração em computação é chamado de abstração de controle. Abstração de controle refere-se à visão de uma estrutura de controle; é a separação da visão lógica de uma estrutura de controle de sua implementação. Uma estrutura de controle nos permite alterar esse fluxo sequencial de controle de um algoritmo. Os comandos **while** e **if** são estruturas de controle. Como essas estruturas de controle são implementadas nas linguagens para as quais podemos traduzir um algoritmo é irrelevante para o design dos algoritmos.

* Abstração de dados: A separação da visão lógica dos dados de sua implementação.

* Abstração procedural: A separação da visão lógica de uma ação de sua implementação.

* Abstração de controle: A separação da visão lógica de uma estrutura de controle de sua implementação.

* Estrutura de controle: Uma instrução usada para alterar o fluxo de controle normalmente sequencial.

Abstração é a ferramenta mais poderosa que as pessoas têm para gerenciar a complexidade. Essa afirmação é verdadeira tanto na computação quanto na vida real.

Nomeando Coisas

Quando escrevemos algoritmos, usamos frases abreviadas para representar as tarefas e os dados com os quais estamos lidando. Damos nomes aos dados e processos. Esses nomes são chamados de identificadores. Por exemplo, usamos **novaBase** e **numeroDecimal** no algoritmo de mudança de base. Também demos nomes às tarefas. Por exemplo, usamos **Dividir** para nomear a tarefa de dividir um array no algoritmo Quicksort. Nossos identificadores para valores de dados foram criados a partir de uma combinação de palavras, usando letras maiúsculas para tornar o significado mais claro. Deixamos os nomes das tarefas como frases. Eventualmente, os nomes das tarefas devem ser convertidos em um único identificador.

Quando chegamos ao estágio em que traduzimos um algoritmo para um programa em uma linguagem que um computador pode executar, podemos ter que modificar os identificadores. Cada linguagem tem suas próprias regras sobre a formação de identificadores. Portanto, há um processo de duas etapas: dados e ações recebem nomes no algoritmo e, em seguida, esses nomes são traduzidos em identificadores que atendem às regras da linguagem de computador. Observe que atribuir identificadores a dados e ações é uma forma de abstração.

Testes

Demonstramos os testes na fase do algoritmo usando tutoriais de algoritmos. Mostramos como projetar planos de teste e implementamos um em linguagem assembly. Os testes são importantes em todas as etapas da programação. Existem basicamente dois tipos de teste: teste de caixa-branca, que se baseia no próprio código, e teste de caixa-preta, que se baseia no teste de todos os valores de entrada possíveis. Frequentemente, um plano de teste incorpora ambos os tipos de teste.

RESUMO

Polya, em seu livro clássico "How to Solve It", delineou uma estratégia de resolução de problemas para problemas matemáticos. Essa estratégia pode ser aplicada a todos os problemas, incluindo aqueles para os quais um programa de computador deve ser escrito. Essas estratégias incluem fazer perguntas, procurar coisas familiares e dividir para conquistar; quando aplicadas, elas devem levar a um plano para a resolução de um problema. Em computação, esse plano é chamado de **algoritmo**.

Distinguem-se duas categorias de laços: controlados por contagem e controlados por eventos. Um laço controlado por contagem executa o laço um número predeterminado de vezes. Um laço controlado por eventos é executado até que um evento dentro do laço mude.

Os dados vêm em duas formas: atômicos (simples) e compostos. Um array é uma estrutura homogênea que dá um nome a uma coleção de itens e permite ao usuário acessar itens individuais por posição dentro da estrutura.

Buscar é o ato de procurar um valor específico em um array. Neste capítulo, examinamos a busca linear em um array não ordenado, a busca linear em um array ordenado e a busca binária em um array ordenado. Ordenar é o ato de colocar os itens de um array em algum tipo de ordem. A ordenação por seleção (selection sort), ordenação por bolha (bubble sort), ordenação por inserção (insertion sort) e ordenação rápida (quicksort) são quatro algoritmos de ordenação comumente usados.

Algoritmos recursivos são algoritmos para os quais o nome de um subprograma aparece no próprio subprograma. O cálculo do fatorial e a busca binária são algoritmos naturalmente recursivos.

EXERCÍCIOS

Para os Exercícios 1 a 6, combine a estratégia de resolução de problemas com a definição ou exemplo.

A. Faça perguntas B. Procure por coisas familiares C. Divida para conquistar

1. A primeira estratégia a ser usada diante de um problema.
2. Não reinvente a roda.
3. Estratégia usada nos algoritmos de busca binária.
4. Uma solução para um problema anterior é apropriada para o problema atual?
5. Estratégia usada no algoritmo Quicksort.

6. Há uma aparente contradição na declaração do problema.

Para os Exercícios 7 a 10, combine a fase a seguir com sua saída.

A. Fase de análise e especificação.	C. Fase de implementação.
B. Fase de desenvolvimento do algoritmo.	D. Fase de manutenção.

7. Programa de trabalho
8. Nenhum
9. Declaração do problema
10. Solução geral

Para os Exercícios 11 a 15, combine o termo com a definição.

A. Ocultação de informações	C. Abstração de dados	E. Abstração de controle
B. Abstração	D. Abstração procedural	

11. A prática de ocultar os detalhes de um módulo com o objetivo de controlar o acesso aos detalhes do módulo.
12. Um modelo de um sistema complexo que inclui apenas os detalhes essenciais para o visualizador.
13. A separação da visão lógica de uma ação de sua implementação.
14. A separação da visão lógica de uma estrutura de controle de sua implementação.
15. A separação da visão lógica dos dados de sua implementação.

Para os Exercícios 16 a 36, marque as respostas como Verdadeira ou Falsa:

16. Laços controlados por contagem se repetem um número específico de vezes.
17. Laços controlados por evento se repetem um número específico de vezes.
18. Laços controlados por contagem são controlados por um contador.
19. Laços controlados por evento são controlados por um evento.
20. Um laço infinito é um laço que nunca termina.
21. Laços podem ser aninhados, mas estruturas de seleção não.
22. Estruturas de seleção podem ser aninhadas, mas laços não.
23. Todas as estruturas de controle podem ser aninhadas.
24. O algoritmo da raiz quadrada usa um laço controlado por contagem.
25. Um array é uma estrutura homogênea, mas um registro não é.
26. Um registro é uma estrutura heterogênea, mas um array não é.
27. Um registro é uma estrutura homogênea; um array é uma estrutura heterogênea.
28. O algoritmo bubble sort envolve encontrar o menor item na porção não ordenada do array e trocá-lo com o primeiro item não ordenado.
29. O quicksort nem sempre é rápido.
30. Uma busca binária pode ser aplicada tanto a um array ordenado quanto a um array não ordenado.
31. Uma busca binária é sempre mais rápida que uma busca linear.
32. Uma ordenação por seleção coloca um item a mais em seu lugar permanente a cada iteração.
33. Uma ordenação por inserção coloca um item a mais em seu lugar em relação à porção já ordenada.
34. Recursão é outra palavra para iteração.
35. Algoritmos recursivos usam instruções IF.
36. Algoritmos iterativos usam instruções WHILE.

Os exercícios 37 a 62 são perguntas de resposta curta.

37. Liste os quatro passos da lista "Como Resolver" de Polya.
38. Descreva os quatro passos listados no Exercício 37 com suas próprias palavras.
39. Liste as estratégias de resolução de problemas discutidas neste capítulo.
40. Aplique as estratégias de resolução de problemas às seguintes situações.
    a) Comprar um brinquedo para seu primo de quatro anos.
    b) Organizar um banquete de premiação para seu time de futebol
    c) Comprar um vestido ou terno para um banquete de premiação no qual você será homenageado
41. Examine as soluções do Exercício 40 e determine três coisas que elas têm em comum.
42. O que é um algoritmo?
43. Escreva um algoritmo para as seguintes tarefas.
    a) Fazer um sanduíche de pasta de amendoim e geleia.
    b) Levantar de manhã
    c) Fazer a lição de casa
    d) Dirigir para casa à tarde
44. Liste as fases do modelo de resolução de problemas computacionais.
45. Como o modelo de resolução de problemas computacionais difere do modelo de resolução de problemas de Polya?
46. Descreva as etapas da fase de desenvolvimento do algoritmo.
47. Descreva as etapas da fase de implementação.
48. Descreva as etapas da fase de manutenção.
49. Procure uma receita de brownie de chocolate em um livro de receitas e responda às seguintes perguntas.
    a) A receita é um algoritmo? Justifique sua resposta.
    b) Organize a receita como um algoritmo, usando pseudocódigo.
    c) Liste as palavras que têm significado em computação.
    d) Liste as palavras que têm significado na culinária.
    e) Prepare os brownies e leve-os ao seu professor.
50. Dissemos que seguir uma receita é mais fácil do que desenvolver uma. Vá ao supermercado e compre um vegetal que você nunca cozinhou (ou comeu) antes. Leve o para casa e desenvolva uma receita. Escreva sua receita e sua crítica do processo. (Se estiver boa, envie aos autores.)
51. Descreva o processo de design top-down.
52. Diferencie uma etapa concreta de uma etapa abstrata.
53. Escreva um design top-down para as seguintes tarefas.
    a) Comprar um brinquedo para seu primo de quatro anos
    b) Organizar um banquete de premiação para seu time de futebol
    c) Comprar um vestido ou terno para um banquete de premiação no qual você será homenageado
54. Escreva um design top-down para as seguintes tarefas.
    a) Calcular a média de dez notas de testes
    b) Calcular a média de um número desconhecido de notas de testes
    c) Descreva as diferenças entre os dois designs
55. Escreva um design top-down para as seguintes tarefas.
    a) Encontrar um número de telefone na lista telefônica
    b) Encontrar um número de telefone na internet
    c) Encontrar um número de telefone em um pedaço de papel que você perdeu
    d) Descreva as diferenças entre esses designs
56. Distinguir entre informação e dados.
57. Escreva um modelo top-down para classificar uma lista de nomes em ordem alfabética.

58. Responda
    a) Por que a ocultação de informações é importante?
    b) Cite três exemplos de ocultação de informações que você encontra todos os dias.
59. Um avião é um sistema complexo.
    a) Apresente uma abstração de um avião da perspectiva de um piloto.
    b) Apresente uma abstração de um avião a partir da perspectiva de um passageiro.
    c) Apresente uma abstração de um avião a partir da perspectiva da tripulação de cabine.
    d) Apresente uma abstração de um avião a partir da perspectiva de um mecânico de manutenção.
    e) Apresente uma abstração de um avião a partir da perspectiva do escritório corporativo da companhia aérea.
60. Liste os identificadores e indique se eles nomearam dados ou ações para os projetos no Exercício 53.
61. Liste os identificadores e indique se eles nomearam dados ou ações para os projetos no Exercício 54.
62. Liste os identificadores e indique se eles nomearam dados ou ações para os projetos no Exercício 55.

Os Exercícios 63–65 usam o seguinte array de valores.

Tamanho
11

Lista										
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
23	41	66	20	2	90	9	34	19	40	99

63. Mostre o estado da lista quando **primeiroNaoOrdenado** é definido pela primeira vez como igual ao quarto item na ordenação por seleção.
64. Mostre o estado da lista quando **primeiroNaoOrdenado** é definido pela primeira vez como igual ao quinto item no algoritmo de ordenação por bolhas.
65. Mostre o estado da lista quando a primeira chamada recursiva é feita no Quicksort usando **lista[0]** como o valor da divisão.

Os exercícios 66 e 67 usam o seguinte array de valores.

Tamanho
11

Lista										
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
5	7	20	33	44	46	48	99	101	102	105

66. Quantas comparações são necessárias usando uma busca sequencial para encontrar os seguintes valores ou determinar que o item não está na lista?
    a) 4 b) 44 c) 45 d) 105 e) 106
67. Quantas comparações são necessárias usando uma busca binária para encontrar os seguintes valores ou determinar que o item não está na lista?
    a) 4 b) 44 c) 46 d) 105 e) 106

QUESTÕES DE REFLEXÃO

1. Distinga entre um programa que a CPU pode executar diretamente e um programa que deve ser traduzido.
2. O design top-down cria um andaime que é usado para escrever um programa. Todo esse andaime é apenas um desperdício de esforço? Ele é usado novamente? Qual o seu valor depois que o programa está em execução?
3. Qual das estratégias de resolução de problemas você mais usa? Você consegue pensar em algumas outras que você usa? Elas seriam apropriadas para a resolução de problemas de computação?
4. Existem vários exemplos comuns de software de código aberto que muitas pessoas usam no dia a dia. Você consegue citar algum?

5. Você acredita que a qualidade de um produto de software de código aberto provavelmente será maior ou menor do que a qualidade do software produzido por uma grande corporação? Como você acha que o suporte técnico para software de código aberto se compara ao de software proprietário?
6. Daniel Bricklin, cuja biografia aparece no Capítulo 12, não patenteou (nem registrou direitos autorais) de seu software, acreditando que ele não deveria ser proprietário. Como resultado, ele perdeu muito dinheiro na forma de possíveis royalties. Você considera suas ações visionárias ou ingênuas?
7. A Free Software Foundation é uma instituição de caridade isenta de impostos que arrecada fundos para o trabalho no Projeto GNU. O software GNU é livre. Leia sobre sua filosofia na Web. Compare os produtos GNU com os de fabricantes como Microsoft e Sun.
8. Se você continuasse com a computação e se tornasse um programador, qual lado do argumento você escolheria: O software deveria ser protegido por direitos autorais ou deveria ser livre?